

Meerkat: Alert Triage for Multi-Detector Security Operations Data

Jiacheng Wang

July 2026

Abstract

Security detectors generate far more alerts than analysts can review, so the operational problem is ordering rather than detection. This report describes Meerkat, a triage pipeline that normalizes alerts from three heterogeneous detectors, ranks them with supervised models, enriches them with MITRE ATT&CK context, and produces a fixed size daily review queue. Evaluation uses leave one scenario out testing across the eight simulated environments of the AIT-ADS dataset. A Random Forest trained on attack window membership attains 86.44% alert level recall at 54.13% workload reduction, yet reaches only 28 of 79 labelled attack windows, with 93% of its decision weight concentrated on activity volume features. A second model trained on sparse per alert event labels reaches 26 windows on its own, while the fused pipeline reaches 20 and fails a pre-registered acceptance threshold of more than 28 windows. Analysis of the raw alerts indicates that the limiting factor is discarded alert local evidence rather than model family. All negative results are reported unchanged.

1 Introduction

A Security Operations Center deploys detectors that convert raw telemetry into alerts. Detection is therefore already complete when an alert exists, and the remaining task is triage: deciding which alerts merit human attention under a fixed review capacity. In the dataset used here a single simulated company produces approximately 6,000 alerts on a day containing no attack, whereas a Tier 1 analyst reviews on the order of tens of alerts per shift.

Meerkat addresses the ordering problem for heterogeneous detector output. Its contributions are a cross detector normalization layer, a feature representation based on temporal context, two supervision strategies derived from different label sources, rule based ATT&CK enrichment, and an evaluation protocol that measures attack window coverage under a fixed analyst budget across unseen environments.

2 Dataset

Experiments use AIT-ADS [1], published by the Austrian Institute of Technology under CC BY 4.0. The dataset contains eight independently simulated enterprise environments, each monitored concurrently by Suricata (network intrusion detection), Wazuh (host intrusion detection) and AMiner (log anomaly detection). Each environment is subjected to a scripted multi step attack comprising network scanning, service scanning, web enumeration, webshell upload, credential cracking, reverse shell, privilege escalation, service disruption and DNS exfiltration.

The raw archive contains 2,655,821 alert records. Suricata alerts are collected by Wazuh and appear twice, once decoded from `eve.json` and once re-parsed from `fast.log`; removing the lower information duplicates leaves 2,349,098 alerts across 79 labelled attack windows.

Two properties motivate this choice of dataset. Three concurrent detectors make cross detector normalization a genuine rather than artificial problem, and eight independent environments permit training on seven while testing on one that the model has never observed.

2.1 Label semantics

Each alert carries two labels produced by different processes. The field `time_label` names the attack phase active when the alert fired, derived from recorded phase intervals. The field `event_label` names the attack event to which the alert could be traced in the underlying logs or network flows. Table 1 shows representative rows from the `russellmitchell` environment.

Table 1: Representative labelled alerts. Both labels are incomplete in different ways.

Time	Alert name	<code>time_label</code>	<code>event_label</code>
00:02:27	Wazuh: ClamAV database update	<code>false_positive</code>	–
03:25:14	Wazuh: ClamAV database update	<code>network_scans</code>	–
03:57:26	Wazuh: Multiple web server 400 errors	<code>dirb</code>	<code>dirb</code>
04:37:40	Wazuh: User successfully changed UID	<code>privilege_escalation</code>	<code>attacker_change_user</code>
04:37:25	AMiner: High entropy in Apache request	<code>reverse_shell</code>	–
00:00:09	AMiner: New ip address in DNS logs	<code>false_positive</code>	<code>dnsteal</code>

The second row is an antivirus update that coincided with attacker scanning, so its phase label conveys temporal proximity without indicating malicious activity. The fifth row occurs inside the reverse shell window yet carries no event label, because the official labelling rules cover only selected log sources. The sixth row carries a `dnsteal` event label while its phase label is `false_positive`, representing genuine exfiltration activity outside the recorded interval.

Aggregate label statistics are given in Table 2. Event labels are strongly concentrated: `dirb` alone supplies 1,688,945 of 1,817,250 positives (92.9%), and the categories `reverse_shell` and `service_stop` have no corresponding event category at all.

Table 2: Joint distribution of the two label sources across all eight environments.

	Inside attack window	Outside
Event labelled	1,745,038	72,212
Not event labelled	13,521	remainder

The 72,212 event labelled alerts falling outside every recorded window establish that absence of a phase label cannot be interpreted as benign. An absent event label likewise denotes unlabelled rather than confirmed benign.

3 Method

3.1 Normalization

The three detectors emit incompatible JSON schemas. Wazuh nests rule metadata under `rule`, Suricata events arrive wrapped inside a Wazuh envelope at `data.alert`, and AMiner uses an `AnalysisComponent` structure with epoch timestamps. Normalization maps all three onto a common table containing detector source, timestamp, alert name, host, native severity, rule identifier and native ATT&CK technique identifiers where present. This corresponds to the field mapping performed by a SIEM when onboarding a log source.

Host identity requires per detector treatment. Wazuh supplies a hostname in `predecoder` for directly parsed logs and falls back to the collecting agent otherwise; Suricata reports a single

sensor; AMiner exposes only the monitored machine’s address, which is resolved to a hostname by evidence gathered from the raw log lines and accepted only when unambiguous.

3.2 Features

Alerts are encoded as 46 numeric columns: one hot indicators for detector and alert name, activity counts over 60, 600 and 3600 second windows computed at host, detector and global scope, time since the previous alert on the same host, a global rarity count for the alert name, a first occurrence indicator for each host and name pair, and a cross detector urgency tier.

The urgency tier reconciles three incompatible native severity scales. Wazuh reports levels 0–15 where higher is more severe, Suricata reports 1–3 where lower is more severe, and AMiner reports no severity. The tier maps each onto a common ordinal scale.

Rare alert names are bucketed before encoding, with the retained vocabulary computed from training data only and reapplied unchanged at prediction time, so that names unseen during training fall into a shared category rather than producing empty indicator columns.

All temporal features are computed strictly backwards. A feature for alert i may use alerts occurring at or before t_i and never afterwards, since future alerts are unavailable at the moment a live alert requires scoring.

3.3 Supervision

Two models are trained on the same feature matrix using different targets.

The *context model* treats any alert carrying a phase label as positive. In Table 1 this includes the antivirus update, so the target expresses temporal coincidence with an attack and only approximates attack relevance.

The *event model* treats any alert carrying an event label as positive, covering rows three, four and six of Table 1 while excluding row five despite its occurrence during the reverse shell. Because an absent label denotes unlabelled rather than benign, the resulting score is consumed as a ranking signal expressing similarity to known attack events, and it is permitted to raise an alert’s priority but never to lower it. Category weights computed within each training fold reduce the dominance of `dirb` without allowing held out data to influence the weighting.

Both models are Random Forests of 300 trees with balanced class weights and a fixed random seed.

3.4 Enrichment and queue construction

Alerts are mapped to ATT&CK techniques and tactics [3] through a reviewed configuration keyed on stable detector rule identifiers, with technique to tactic relationships resolved from the official ATT&CK STIX distribution. The mapping is deliberately rule based so that every claim remains auditable and correctable by an analyst; one vendor supplied mapping is overridden where it was found to be incorrect.

The queue applies a fixed capacity of 50 reviews per UTC day. Alerts are selected in priority band order, and within each band one representative per repeated host, detector and rule group is selected before remaining repetitions, so that a single noisy rule cannot consume the daily capacity while quiet repeated activity still surfaces.

4 Evaluation protocol

Evaluation uses leave one scenario out testing: seven environments provide training and validation data while the eighth is held out entirely, repeated eight times. Host identity is excluded

from the model to limit environment memorization, the categorical vocabulary and any weighting are fitted within each fold, and rolling context is computed per environment so that no window spans two environments.

The primary operational metric is the number of labelled attack windows reached by the queue at a fixed budget of 50 reviews per day. Alert level recall, workload reduction, outside window review rate and per category event recall are reported as secondary measures. An acceptance threshold of more than 28 reached windows was fixed before the comparison was executed.

5 Results

5.1 Context supervision

Table 3 reports the scenario macro results of the context model.

Table 3: Context model, leave one scenario out, scenario macro means.

Metric	Result
Attack window alert recall	86.44%
Workload reduction	54.13%
Outside window review rate	0.011%
Average precision	98.87%
Worst environment recall (Shaw)	44.70%
Best environment recall (Wheeler)	99.79%

These figures overstate the model when read alone. The same model reaches only 28 of 79 labelled attack windows, and approximately 93% of Random Forest importance falls on temporal volume features. Alert level recall is dominated by two high volume phases, **dirb** and **wpscan**, which contribute thousands of alerts each, whereas quiet phases contribute single digit counts. The model therefore produces an ordering by alert volume, which is useful for noisy intrusion stages and uninformative for quiet ones.

A window ablation over 60, 600 and 3600 second scopes confirmed that temporal context is nonetheless necessary: removing all window features raises the validation outside window review rate from 0.13% to 29.03% and lowers workload reduction from 22.91% to 16.49%. Individual window lengths are strongly correlated, and no reduced subset offered a practical improvement, so the full configuration was retained.

Compared against simple baselines, native detector urgency achieves higher alert recall (95.52%) at a substantially higher outside window review rate (14.83%), indicating that the advantage of the learned model lies in queue concentration rather than raw recall.

5.2 Dual evidence comparison

Table 4 compares all variants at the fixed budget of 50 reviews per day.

Event supervision produces a stronger ranking than context supervision, and the event model improves category macro recall over the context model in all eight held out environments. The fused pipeline nevertheless scores below its own strongest component. Score fusion by maximum allows the dense context signal to displace event ranked alerts from a capacity limited queue, costing six windows relative to the event model alone. The pre-registered threshold of more than 28 windows was not met, and the result of 20 is reported without modification.

Table 4: Attack windows reached at 50 reviews per day, summed over eight held out environments (79 windows total).

Variant	Windows reached
Event model	26
Native detector severity	21
Full hybrid pipeline	20
Event and context fused	20
Native protected	19
Context model	14

5.3 Coverage overlap

Per window analysis at the same budget decomposes coverage as follows: both models reach 12 windows, the event model alone reaches 14 further windows, the context model alone reaches 2, and 51 windows are reached by neither. The two windows recovered only by context supervision are `service_stop` instances in the Fox and Wheeler environments, each containing two source alerts, which is insufficient evidence to establish a general discovery role for context supervision.

Every `network_scans`, `privilege_escalation`, `reverse_shell`, `webshell` and `cracking` window remains unreached by both models.

6 Discussion

The failure is informative in a specific way. The event model demonstrates that per alert supervision ranks better than interval supervision, while the fusion result demonstrates that combining evidence by maximum is inadequate when capacity is fixed and one signal is dense.

Inspection of the raw records indicates that the binding constraint is feature information rather than model family. Normalization currently discards fields that are present in the source data and directly discriminative: Wazuh web alerts carry source addresses, request paths and status codes; privilege escalation alerts carry source and destination usernames, the working directory and the executed command; auditd alerts carry executable paths and process identifiers; Suricata alerts carry addresses, ports, protocol and HTTP metadata. One alert in the dataset records a `sudo` to root executed from the webshell upload directory, which an analyst interprets immediately and which the model cannot observe because the relevant fields never reach the feature matrix.

Quiet phases are additionally constrained by label availability. The categories `reverse_shell` and `service_stop` have no event labels at all, so no supervision signal for them exists regardless of model capacity. These two limitations are distinct: alert local enrichment can address the first, whereas the second requires additional labelled evidence.

7 Limitations

Results derive from eight simulated environments produced by a single generator with a shared attack script, so they constitute development evidence rather than a claim of real world performance. Alerts outside a labelled window are unlabelled rather than confirmed benign, hence the outside window review rate is reported as such and never as a false positive rate. Attack prevalence ranges from 10.2% to 91.2% across environments, far from operational conditions, so workload reduction figures are benchmark specific. Model output is a ranking score and is not calibrated as a probability; the reliability curve is non-monotonic in the middle score range.

8 Conclusion

Meerkat normalizes heterogeneous detector output, ranks alerts under two supervision regimes, enriches them with auditable ATT&CK context and produces a capacity limited review queue evaluated across unseen environments. Interval supervision yields high alert level recall while covering few distinct attack windows; per alert event supervision covers substantially more windows; naive fusion of the two degrades coverage below the better component. The evidence indicates that discarded alert local fields, rather than model family, currently limit performance, and the next step is enrichment of the alert representation with source addresses, user and process context, followed by reconsideration of the fusion rule.

References

- [1] M. Landauer, F. Skopik and M. Wurzenberger, *Introducing a New Alert Data Set for Multi-Step Attack Analysis*, Cyber Security Experimentation and Test Workshop (CSET), 2024.
- [2] M. Landauer, F. Skopik, M. Frank, W. Hotwagner, M. Wurzenberger and A. Rauber, *Maintainable Log Datasets for Evaluation of Intrusion Detection Systems*, IEEE Transactions on Dependable and Secure Computing, 2023.
- [3] MITRE, *MITRE ATT&CK Enterprise Matrix and STIX Distribution*, <https://attack.mitre.org>.
- [4] SophosAI, *That Escalated Quickly: An ML Framework for Alert Prioritization*, arXiv:2302.06648, 2023.
- [5] K. Veeramachaneni, I. Arnaldo, V. Korrapati, C. Bassias and K. Li, *AI²: Training a Big Data Machine to Defend*, IEEE International Conference on Big Data Security, 2016.
- [6] T. van Ede et al., *DeepCASE: Semi-Supervised Contextual Analysis of Security Events*, IEEE Symposium on Security and Privacy, 2022.